

C++ Training Project

Aminata BARRY - Anjali PANARAKOTTIL VIJAYAN
Master 2 IRFA

Université Paris 1- Panthéon Sorbonne

10/02/2026

Table des matières

1	Numerical Integration Methods and Code Architecture	2
1.1	Implemented Numerical Methods	2
1.2	Benchmark on Test Functions	2
1.3	Choice of Meta-Parameters	2
1.4	Code Architecture	3
2	Extension to Two-Dimensional Integration	3
2.1	Numerical Integration of 2D Functions	3
2.2	Tests and Observed Behaviour	4
3	Use of Artificial Intelligence	4

1 Numerical Integration Methods and Code Architecture

Several numerical integration methods were implemented in this project in order to approximate definite integrals of the form

$$\int_a^b f(x) dx.$$

Each method is implemented in a dedicated C++ class, following an object-oriented design based on inheritance and polymorphism.

1.1 Implemented Numerical Methods

Trapezoidal Rule The trapezoidal rule approximates the integral by replacing the function with a piecewise linear interpolation over a uniform discretization of the interval $[a, b]$. The integral is then approximated by the sum of trapezoid areas. This method is simple but may suffer from large errors when the function is not smooth.

Simpson's Rule Simpson's rule improves accuracy by approximating the function with quadratic polynomials on each pair of subintervals. It requires an even number of discretization points and generally provides much higher accuracy for smooth functions compared to the trapezoidal rule.

Gaussian Quadrature The Gaussian quadrature method uses a weighted sum of function evaluations at carefully chosen points inside each subinterval. In this project, a two-point Gauss-Legendre quadrature is applied on each subinterval, leading to very high accuracy even with a relatively small number of intervals.

Monte-Carlo Method The Monte-Carlo method is a stochastic approach based on random sampling. The integral is approximated using the probabilistic formula

$$\int_a^b f(x) dx \approx \frac{b-a}{n} \sum_{i=1}^n f(X_i),$$

where the X_i are independent random variables uniformly distributed on $[a, b]$. This method is robust and applicable to functions with singularities, but it converges slowly with an error of order $O(1/\sqrt{n})$.

1.2 Benchmark on Test Functions

The implemented methods were benchmarked using several test functions provided in the project statement : $x^2 \cos(x)$, x^{10} , $x^{-1/2}$ and $\log(x)$ on the interval $[0, 1]$. Exact values are known and were used to compute the numerical errors.

For smooth functions such as $x^2 \cos(x)$ and x^{10} , Simpson's rule and Gaussian quadrature achieve near machine-precision accuracy, while the trapezoidal rule converges more slowly. The Monte-Carlo method produces reasonable approximations but with a noticeably larger statistical error.

For functions presenting singularities at the lower bound, such as $x^{-1/2}$ and $\log(x)$, deterministic methods based on uniform discretization fail and diverge. In contrast, Gaussian quadrature and Monte-Carlo integration remain stable and provide meaningful approximations, illustrating the robustness of these approaches.

1.3 Choice of Meta-Parameters

The number of discretization intervals was chosen as $n = 1000$ for the trapezoidal and Simpson methods in order to ensure sufficient accuracy while keeping the computational cost reasonable. Simpson's rule internally adjusts this value to enforce an even number of intervals.

For Gaussian quadrature, fewer intervals are required due to the higher order of accuracy of the method. A smaller number of subintervals was therefore sufficient to achieve very precise results.

For the Monte-Carlo method, a large number of random samples (up to 10^6) was used. This choice is motivated by the slow convergence rate of Monte-Carlo integration, where the error decreases proportionally to $1/\sqrt{n}$. Increasing the number of samples significantly improves the statistical accuracy of the results.

1.4 Code Architecture

The software architecture of this project was designed with a strong emphasis on object-oriented programming principles, in order to clearly separate concerns, avoid code duplication, and ensure extensibility. The core of the design relies on abstraction, inheritance, and runtime polymorphism.

Two abstract base classes form the foundation of the architecture. The **Function** class represents a mathematical function to be integrated. It defines a pure virtual overload of the function call operator, which allows any concrete function to be evaluated using a uniform interface. This design choice makes it possible to pass functions as objects and to evaluate them independently of their concrete implementation.

Similarly, the **Solver** class is an abstract base class defining a common interface for all numerical integration methods through the pure virtual method **integrate**. Each numerical method is implemented as a concrete class derived from **Solver**. This ensures that all solvers share the same interface while allowing different numerical strategies to be implemented independently.

The use of polymorphism allows solvers to operate on any function derived from **Function** without knowing its concrete type. As a result, the numerical integration algorithms are completely decoupled from the mathematical functions they process. This separation of responsibilities improves code readability, testability, and maintainability.

Each solver class encapsulates its own meta-parameters, such as the number of discretization intervals or the number of Monte-Carlo samples. These parameters are stored as private data members and initialized through constructors, enforcing proper encapsulation and preventing unintended modification during execution.

The architecture also facilitates extensibility. New numerical methods can be added by simply creating a new class derived from **Solver**, without modifying existing code. Similarly, new mathematical functions can be introduced by deriving from **Function**. This design follows the open/closed principle, where the code is open for extension but closed for modification.

Overall, this object-oriented structure provides a clear and flexible framework that highlights the use of abstraction and polymorphism in C++, while remaining well suited for numerical experimentation and comparison of integration methods.

2 Extension to Two-Dimensional Integration

2.1 Numerical Integration of 2D Functions

As an extension of the one-dimensional integration library, the code was generalized to approximate double integrals of the form

$$\int_a^b \int_c^d f(x, y) dy dx,$$

for functions $f : \mathbb{R}^2 \rightarrow \mathbb{R}$. The approach is based on repeated one-dimensional integration : for each fixed value of x , the inner integral in y is approximated using an existing one-dimensional solver, and the resulting values are then integrated with respect to x using a composite trapezoidal rule.

To represent bivariate functions, an abstract base class **Function2D** was introduced, in analogy with the **Function** class used in the one-dimensional setting. It defines a pure virtual overload of the call operator with two arguments, allowing any concrete two-dimensional function to be evaluated through a common interface. Example implementations include the functions

$$f(x, y) = xy \quad \text{and} \quad f(x, y) = \sin(\pi x) \sin(\pi y),$$

which were used as test cases.

On the solver side, a new abstract base class **Solver2D** was defined to represent two-dimensional integration methods. A concrete class **Repeated1DSolver2D** implements the repeated integration strategy described above. For a given function f , the inner integral

$$g(x) = \int_c^d f(x, y) dy$$

is approximated by wrapping $f(x, \cdot)$ into a one-dimensional **Function** object and passing it to an existing one-dimensional solver (for example Simpson's rule) on the interval $[c, d]$. The outer integral $\int_a^b g(x) dx$ is then approximated by a composite trapezoidal rule on a uniform grid in x . This design demonstrates how the original one-dimensional infrastructure can be reused and extended to higher dimensions with minimal code duplication.

2.2 Tests and Observed Behaviour

The two-dimensional framework was validated on simple test functions with known exact integrals. A first example was

$$f(x, y) = x y \quad \text{on} \quad [0, 1] \times [0, 1],$$

for which

$$\int_0^1 \int_0^1 x y \, dy \, dx = \frac{1}{4}.$$

Using Simpson's rule for the inner integration in y and the composite trapezoidal rule for the outer integration in x , the numerical result was very close to the exact value, with an absolute error decreasing as the number of discretization points in each direction was increased.

A second test involved a more oscillatory function, such as $\sin(\pi x) \sin(\pi y)$ on $[0, 1]^2$, whose integral vanishes. In this case as well, the repeated one-dimensional method produced accurate approximations, provided that the discretization was fine enough to resolve the oscillations in both directions. These experiments confirm that the architecture designed for one-dimensional integration can be effectively reused for two-dimensional problems, and that classical quadrature rules remain reliable in the tensor-product setting.

3 Use of Artificial Intelligence

Artificial Intelligence tools were used during this project as a support for understanding theoretical concepts, debugging C++ code, and improving the overall structure of the implementation. In particular, AI was used to clarify object-oriented design choices, verify the correctness of numerical formulas, and assist in the interpretation of numerical results.

The final design, implementation, and validation of the code were carried out by the authors. Artificial Intelligence was used as a complementary tool to enhance learning and productivity, not as a substitute for personal understanding or development.